

הרצאה 9 – Client-Server

בל הזכויות שמורות לקרן יעקב 2025

תוכן עניינים

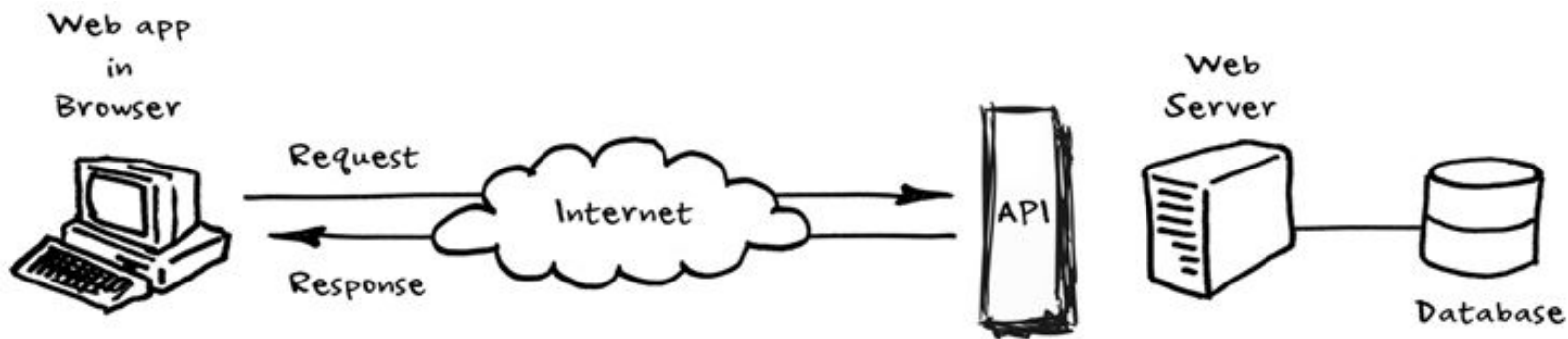
1. Client – Server: REST, XML, JSON
2. AJAX, fetch, promises
3. HTTP Methods & RESTful API



1. Client-server: REST, XML, JSON

Client-Server

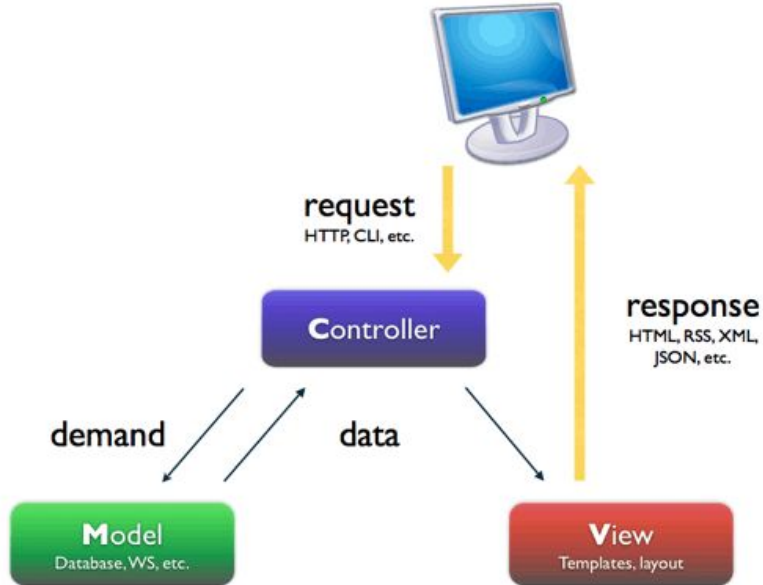
באופן כללי, התקשורת בין צד הלקוח לצד השרת נעשה ע"י בקשות ותשובות HTTP



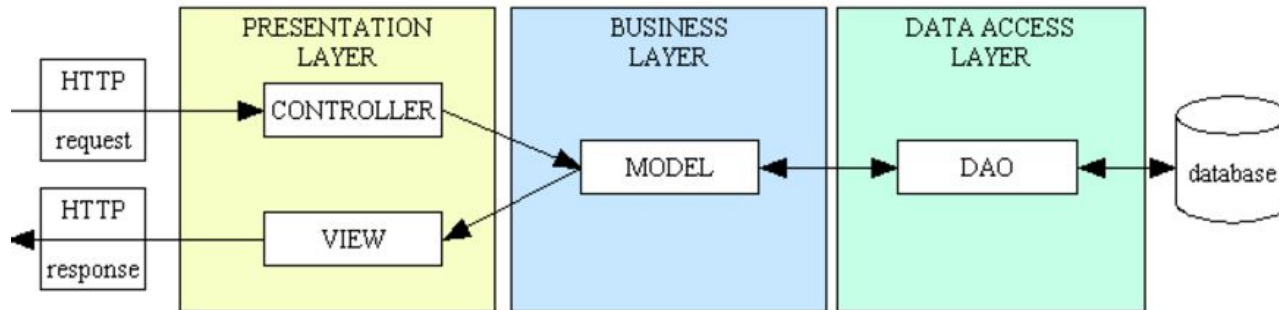
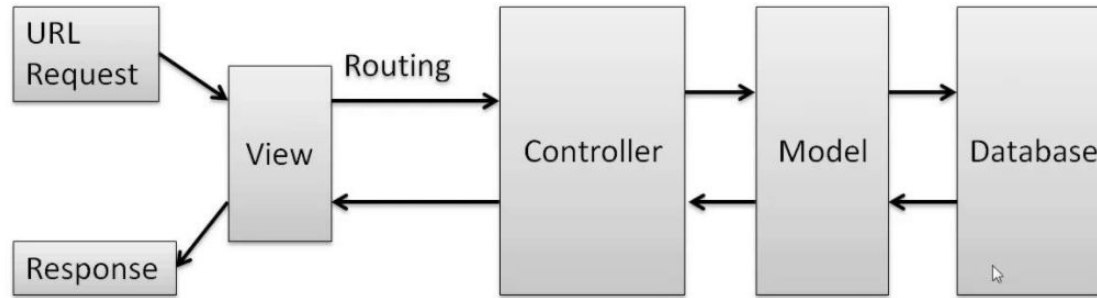
MVC

● הפרדת השכבות המסורתית - חלוקה לפי Separation of concerns

● בפרוייקט שלנו, מי M-V-C ?



MVC



REST = Representational State Transfer

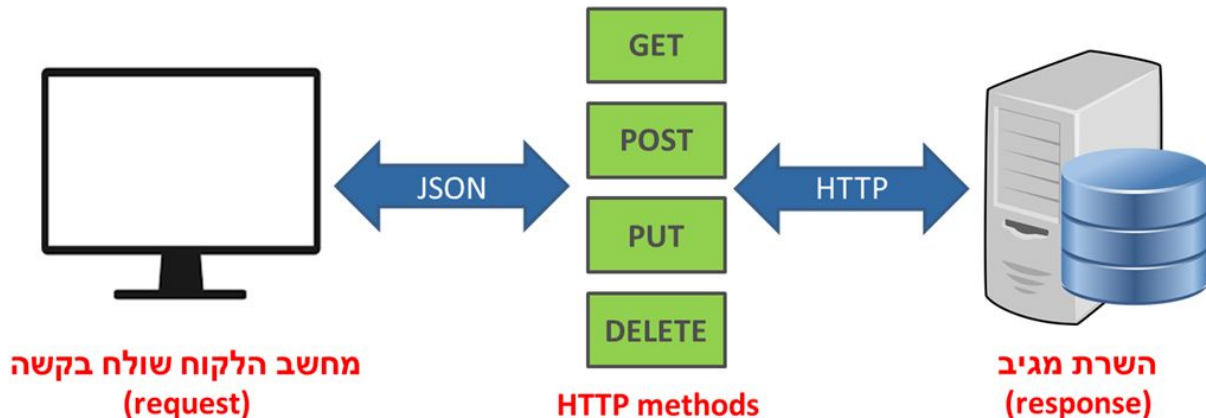
● ארכיטקטורת REST עובדת מהר יותר בגלל עיבוד מינימלי ויעיל יותר

● REST שולח הודעות על גבי פרוטוקול HTTP

● מצפה לקבל URI עם פרמטרים או אובייקטי JSON, בהתאם לאופן שבו ה- web service תוכנת

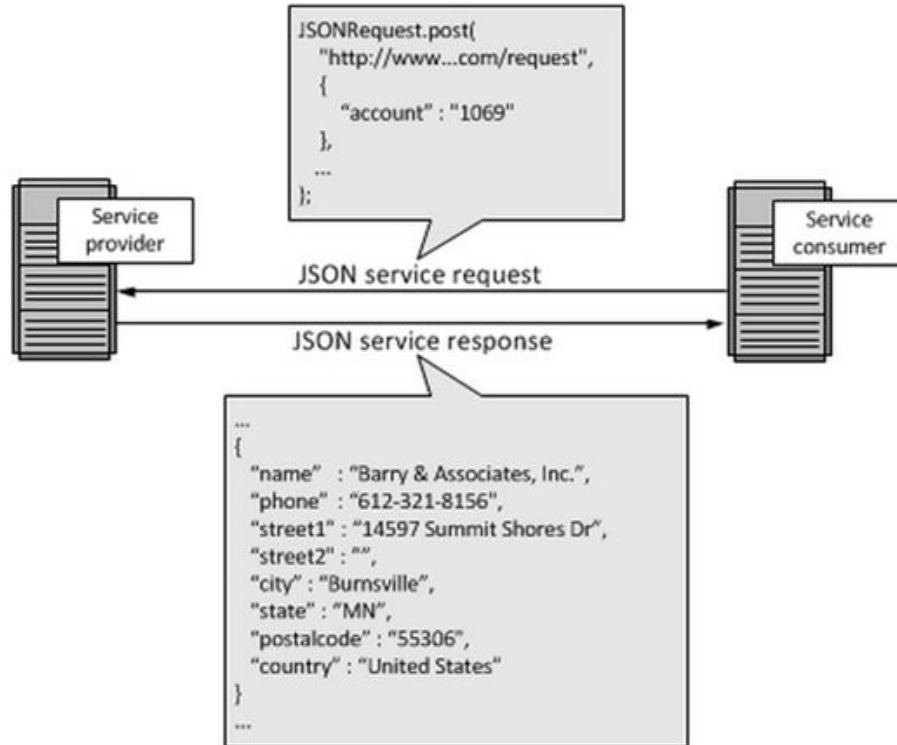
● בדרך כלל, מחזיר תשובה בצורת XML או JSON (נדבר בהמשך)

● JSON יותר מקובל בימינו - הוא קטן, יעיל, מינימליסטי



Restful web service

● שירות REST עם JSON



HTTP Message

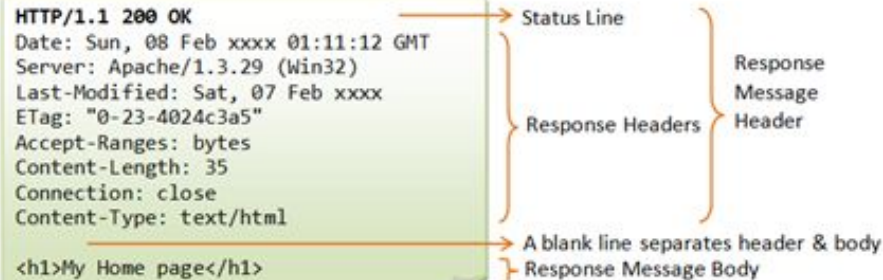


- הודעת request: נשלחת בשליחת בקשה לשרת
- מכילה שורת בקשה + האדרים של הבקשה + ההודעה עצמה
- שדות הבקשה request header fields - הגדרה בצד לקוח
- הודעת response: מתקבלת בתגובה מהשרת
- מכילה סטטוס + האדרים של התגובה + ההודעה עצמה
- שדות התגובה response header fields - הגדרה בצד שרת
- השליחה לשרת מתחילה באחד מהפעלים: GET, POST וכו'

HTTP Message

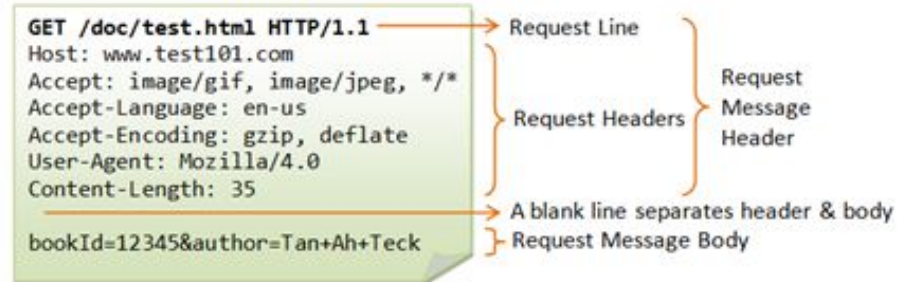
Response

הודעה שנשלחת בחזרה ללקוח



Request

הודעה שנשלחת לשרת ב- GET



XML



- eXtensible Markup Language
- הגדרת תגיות עצמאיות לפי צרכינו
- ה-XML מציג תוכן, נתונים, ולא עיצוב
- אופטימלי לאחסון קטן של נתונים, אחרת כמובן שמומלץ להיעזר בבסיס נתונים
- XML שומר ומחזיק נתונים בלוגיקה שאנחנו מכירים מעולם בסיסי הנתונים

- דוגמה לקובץ xml:

```
Shenkar2024Keren > 7.1-REST > demo.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <Circles>
3      <Circle id="227" name="Dan" img="profileD.jpg"></Circle>
4      <Circle id="231" name="Ben" img="benImg.jpg"></Circle>
5      <Circle id="236" name="Gal" img="gals.jpg"></Circle>
6  </Circles>
```

XML - המשך

● בתחילת המסמך יש שורת הצהרה: `<?xml version="1.0" encoding="utf-8"?>`

● כל תגית חייבת להיסגר, תגיות ללא תוכן תסגורנה את עצמן

● מאפיינים של תגיות חייבים להיות בפורמט: `some="thing"`

● חוקיות סגירת תגיות לפי חוקי סגירת סוגריים במתמטיקה

● רגישות לאותיות גדולות וקטנות - case sensitive

● מוסכמות:

○ מאפיינים באותיות קטנות

○ תגיות לשיקולנו, העיקר שנהיה עקביים

○ מקובל שתגית מרכזית Items מכילה הרבה תגיות Item

Shenkar2024Keren > 7.1-REST > demo.xml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <Circles>
3   <Circle id="227" name="Dan" img="profileD.jpg"></Circle>
4   <Circle id="231" name="Ben" img="benImg.jpg"></Circle>
5   <Circle id="236" name="Gal" img="gals.jpg"></Circle>
6 </Circles>
```

JSON - תזכורת

- מוגדר באובייקט מסוג json (ראשי תיבות של JavaScript Object Notation)
- ניתן להמרה למחרוזת בעזרת stringify או לאובייקט בעזרת parse
- משמש לשמירת נתונים מסוגים שונים, לשליחה על גבי http בין דפים/שרתים
- ל-json יש מבנה של key-value
- דוגמה לקובץ json:

Shenkar2024Keren > 6.1-JS-lec2 > json-example.json > ...

```
1  {
2    "menu": {
3      "id": "123",
4      "value": "File",
5      "popup": {
6        "menuitem": [
7          {"value": "New", "onclick": "CreateNewDoc()"},
8          {"value": "Open", "onclick": "OpenDoc()"},
9          {"value": "Close", "onclick": "CloseDoc()"}
10       ]
11     }
12   }
13 }
```

JSON

- אחת השיטות להחזקת ולשיתוף מידע מהיר, קל ויעיל
- התחיל ב-JavaScript, אך כיום מהווה פורמט עצמאי ולכן בשימוש גם על ידי C ושפות נוספות
- מהווה מחליף ל-XML ברוב המקרים בימינו
- ניתן להשתמש בו כמשתנה או כקובץ, מחייב טעינה מקובץ לתוך משתנה (תחליף לדטבייס)
- מבנה דאטה לא רלציוני NoSQL, לדוגמה, Key-Value store

יש להקפיד על כתיבה מסודרת וקריאה, לפי הפורמט (אפשר לבדוק בוולידטור - <https://jsonlint.com>)

```
var student = {  
  "name": "dan sagiv",  
  "address": "Haifa",  
  "grades": [  
    {"math" : 95},  
    {"english" : 90},  
    {"statistics": 85}  
  ]};
```

```
console.log(student.name); //dan sagiv  
console.log(student.grades.length); //3  
console.log(student.grades[0].math); //95
```

- חייבים לשים מרכאות סביב המפתחות והערכים "dd"
- הערכים יכולים להיות:
Strings, Objects, Numbers, Booleans, Array, null

- בסמסטר זה אנחנו עושים רק GET ל-JSON (בסמסטר הבא גם SET)



2. AJAX, fetch, promises

AJAX



- Asynchronous Javascript And XML
- נעזרנו שנים ב- AJAX כדי לקרוא את ה- XML בעזרת קוד javascript
- AJAX מאפשר להביא דאטה מהשרת ללא טעינת הדף מחדש
לדוגמה: חיפוש בגוגל autocomplete
בזמן שאנחנו מחפשים, מגיע דאטה רלוונטי מהשרת
- Javascript יוצר בקשה request לשרת
- Javascript מקבל תגובה response עם הנתונים מהשרת
- אנחנו נשתמש בפונקציה fetch כדי לבצע בקשת AJAX פשוטה מהשרת

אסינכרוניות

- הדאטה מגיע באופן אסינכרוני, ברקע, המשתמש לא מחכה לו
- הדאטה יכול להגיע בצורות xml, אובייקט JSON, קובץ טקסט, html או אחר
- את הדאטה קוראים בעזרת אובייקט XHR (=XML HTTP Request)
- XMLHttpRequest משמש לאינטראקציה עם השרת, והוא לא משמש רק להחזרת xml מהשרת (למרות השם)
- <https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest>

Example

```
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        // Typical action to be performed when the document is ready:
        document.getElementById("demo").innerHTML = xhttp.responseText;
    }
};
xhttp.open("GET", "filename", true);
xhttp.send();
```

0	UNSENT
1	OPENED
2	HEADERS_RECEIVED
3	LOADING
4	DONE

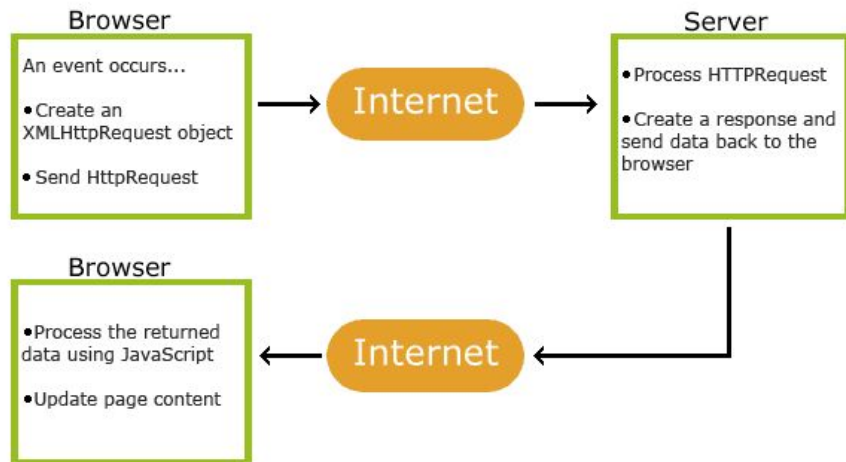
0: request not initialized
1: server connection established
2: request received
3: processing request
4: request finished and response is ready

הסטטוסים מתעדכנים מגרסה לגרסה אבל המספרים נשארים

No http

- ללא שימוש בשרת, הדאטה יטען אבל לא ניתן לקרוא אותו, אלא אם נעלה גם את ה- html לשרת
- קריאת AJAX חייבת להישלח לשרת ולקבל תשובה מהשרת, כך שבלי שרת אי אפשר לבצע את הקריאה
- קבצי ה- html,js וקובץ הדאטה חייבים לשבת באותו דומיין

How AJAX Works



Fetch & Promise



- JS הוא אסינכרוני ומבוסס אירועים
- הוא צריך שמשתמש או מערכת ההפעלה יגרמו/ירו אירוע
- אז אם הוא צריך לקבל תשובה כלשהי הוא לא יחכה, יש מנגנון שלם של עבודה אסינכרונית שלא ניכנס אליה במסגרת הקורס הזה
- Callbacks
- Promises
- נדבר בקצרה על promises בשביל ההבנה

Promises

אובייקט שמייצג את התוצאה שעדיין-לא-מוכנה מפעולה אסינכרונית

שלושה מצבים אפשריים:

A `Promise` is in one of these states:

- *pending*: initial state, neither fulfilled nor rejected.
- *fulfilled*: meaning that the operation was completed successfully.
- *rejected*: meaning that the operation failed.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

```
fetch("data/books.json")  
  .then(response => response.json())  
  .then(data => console.log(data));
```

הפונקציה `fetch` (במימוש הכי פשוט שלה) מקבלת נתיב

ומחזירה אובייקט `promise`,

מוציאים מהתשובה שהתקבלה (`http response`) את ה- `json body`

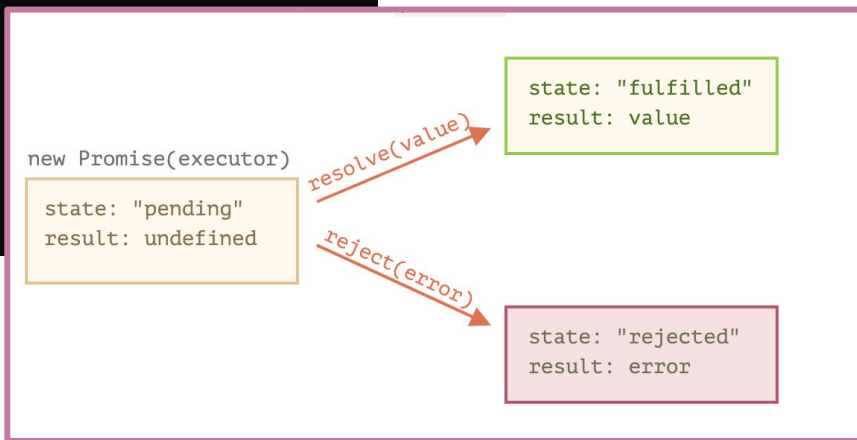
שגם מחזירה אובייקט `promise`

ולבסוף מדפיסים לקונסול את התוכן

Promises

- חדש ב-ES6 - ה-Promise הוא אובייקט המייצג את השלמתה (או כישלונה) של פעולה א-סינכרונית ואת הערך שלה.

```
var promise = new Promise(function(resolve, reject) {  
  // do a thing or twenty  
  if (/* everything turned out fine */) {  
    resolve("Stuff worked!");  
  }  
  else {  
    reject(Error("It broke"));  
  }  
});
```



- הבנאי של ה-Promise מקבל 2 ארגומנטים וקורא לאחד מהם, לפי התוצאה.



3. HTTP METHODS & RESTful API

C
R
U
D



Create



POST



Read



GET



Update



PUT



Delete



DELETE

edureka!

HTTP
Methods

Web API – Architecture



- API = Application Programming Interface
- התגובה מהשרת יכולה לכלול שליחת דף html:
השרת מגיב ללקוח ושולח אליו בחזרה את מבנה העמוד עם תוכנו.
- התגובה מהשרת יכולה לכלול שליחת נתונים:
ללא נראות וללא התנהגות, רק דאטה במבנה מסוים קבוע מראש
- מי קורא את הדאטה הזה?
בקשות ותגובות request and response message ברוב מועברות בפרוטוקול Http

Web API – RESTful Services

חייבים להגדיר מראש:

- אילו שאלות אפשר לשאול את ה-WB? איך מתקבלות התשובות?
- מה פורמט הודעת השאלה? מה פורמט הודעת התשובה? SOAP, XML, JSON...??
- מה ה-syntax? מה שמות הפונקציות, משתנים וסוגי משתנים שבהם עושים שימוש?
- האם נדרש אימות: authentication?
- מה שיטת השליחה? HTTP verbs: POST, GET, PUT, PATCH, DELETE
- API צריך להיות מתועד כך שמתכנתים יוכלו לכתוב את הקוד
- כל עוד ה-API מתועד כהלכה, אפשר לכתוב בכל שפה שרוצים
- לא להתבלבל, זו לא חלוקה לשפת שרת ולשפת לקוח
- גם אפליקציה שכתובה בדוט נט (web server) יכולה לתקשר עם RESTful service שכתוב ב-PHP.

Web API – RESTful Services



● בתור מתכנתי ה-DCS:

- צריך לדעת להפריד מודולים שיעבדו עצמאית וחיצונית, כאילו מישהו אחר כתב אותם.
- לדעת לתת שירות למישהו אחר שאני כמתכנת לא מכיר
- לא מעניין באיזו שפה כתוב הקוד ששואל את השאלות

● בתור מי שעושה שימוש ב-DCS שמישהו אחר כתב:

- צריך לדעת איך לתקשר עם DCS רצוי $\leq$ API
- לא צריך לדעת באיזו שפת תכנות הוא כתוב
- לא צריך לדעת באיזה DB הוא משתמש

בקשות ותשובות של הדפדפן

The screenshot displays the Chrome DevTools Network tab. The top toolbar includes icons for Elements, Console, Sources, Network (selected), Performance, Memory, Application, Security, Lighthouse, and ADBlock. Below the toolbar, there are checkboxes for 'Preserve log' (checked) and 'Disable cache' (unchecked), along with a dropdown menu set to 'Online'. A filter bar shows 'Filter' and 'Hide data URLs' (unchecked), with tabs for 'All' (selected), 'XHR', 'JS', 'CSS', 'Img', 'Media', 'Font', 'Doc', 'WS', 'Manifest', 'Other', 'Has blocked cookies', and 'Blocked'. A timeline at the top shows a duration from 500 ms to 4000 ms. The main panel lists three network requests: 'admin-ajax.php?action=wpsr_share_count', 'collect?t=dc&aip=1&r=3&v=1&v=j86&ti...', and 'content.css'. The 'admin-ajax.php' request is selected, and its details are shown in the right pane. The 'General' tab is active, displaying the following information:

- Request URL:** http://www.sthint.com/wp-admin/admin-ajax.php?action=wpsr_share_count
- Request Method:** POST
- Status Code:** 200 OK
- Remote Address:** 66.96.147.110:80
- Referrer Policy:** no-referrer-when-downgrade

Below the general information, there are expandable sections for 'Response Headers (18)', 'Request Headers (11)', 'Query String Parameters' (with links for 'view source' and 'view URL encoded'), and 'Form Data (1)'. The 'Query String Parameters' section shows 'action: wpsr_share_count'.

בקשות ותשובות של הדפדפן



- פנייה מדפדפן לשרת - נשלחת ב- GET מה- URL או בכל שיטה אחרת דרך קוד
 - מאחר וכל שליחת בקשה מהדפדפן תיעשה בשיטת GET כדי לשלוח בשיטה אחרת יש להיעזר באחד מה- Plugins הקיימים:
- **POSTMAN**
- Advanced REST client
- REST CONSOLE
- פנייה בעזרת טופס לשרת - יכולה להיות ב- GET או POST

בקשות ותשובות של REST

Route name	URL	HTTP Verb	Description
Index	/blogs	GET	Display a list of all blogs
New	/blog/new	GET	Show form to make new blogs
Create	/blogs	POST	Add new blog to database, then redirect
Show	/blogs/:id	GET	Show info about one blogs
Edit	/blogs/:id/edit	GET	Show edit form of one blog
Update	/blogs/:id	PUT	Update a particular blog, then redirect
Destroy	/blogs/:id	DELETE	Delete a particular blog, then redirect

מערכות Restful מתקשרות על גבי פרוטוקול HTTP ע"י HTTP Verbs

אלו אותן שיטות שבהן שולחים פרמטרים בין דפי אינטרנט

לא כל הדפדפנים תומכים בכל השיטות

בולם תומכים לפחות ב-GET ו-POST, הרוב תומכים ב:

● GET לקבלת מידע

● POST להוספת מידע

● PUT לעדכון מידע

● DELETE למחיקת מידע

Task	Method	Path
Create a new customer	POST	/customers
Delete an existing customer	DELETE	/customers/{id}
Get a specific customer	GET	/customers/{id}
Search for customers	GET	/customers
Update an existing customer	PUT	/customers/{id}

● אלו המתודות הכי נפוצות ושימושיות

● ב- Restful services יש לפעלים משמעות

● חובת המפתח להכיר את ה- service שמולו הוא מתכנת

API Documentation – Google Drive example

דוגמה – נסתכל על המבנה של הבקשות שהשרת מצפה לקבל:

Google Drive API > v2

Search

Resource summary

- Files
- About
- Changes
- Children
- Parents
- Permissions
- Revisions
- Apps
- Comments
- Replies
- Properties
- Channels
- Drives
- Standard features

MISC. REFERENCE

Replies

For Replies Resource details, see the [resource representation](#) page.

Method	HTTP request	Description
URIs relative to https://www.googleapis.com/drive/v2 , unless otherwise noted		
delete	DELETE /files/ <i>fileId</i> /comments/ <i>commentId</i> /replies/ <i>replyId</i>	Deletes a reply.
get	GET /files/ <i>fileId</i> /comments/ <i>commentId</i> /replies/ <i>replyId</i>	Gets a reply.
insert	POST /files/ <i>fileId</i> /comments/ <i>commentId</i> /replies	Creates a new reply to the given comment.
list	GET /files/ <i>fileId</i> /comments/ <i>commentId</i> /replies	Lists all of the replies to a comment.
patch	PATCH /files/ <i>fileId</i> /comments/ <i>commentId</i> /replies/ <i>replyId</i>	Updates an existing reply.
update	PUT /files/ <i>fileId</i> /comments/ <i>commentId</i> /replies/ <i>replyId</i>	Updates an existing reply.

קווים מנחים

- בקשת וקבלת JSON
- שימוש בשמות עצם ולא בפעלים, למשל /articles/
- שימוש בצורת הרבים ולא היחיד
- שימוש ב-nesting אם יש אובייקט היררכי, למשל /articles/:id/comments/ (ואם אין היררכיה ביניהם ?)



Create a user
POST /users



Get user information
GET /users/:id



Update a user
PUT /users/:id



Delete a user
DELETE /users/:id

תרגיל כיתה



בואו נתכנן ביחד Restful API.

יש לנו מאגר של משתמשים באפליקציה שלנו. רק משתמש רשום יכול לבצע פעולות. האפליקציה כוללת רשימות השמעה לכל משתמש ובכל רשימת השמעה יש שירים.

עלינו לתמוך בפעולות הבאות:

1. קבלת כל רשימות ההשמעה של משתמש רשום
2. מחיקת שיר מרשימת השמעה של משתמש
3. הוספת שיר לרשימת השמעה קיימת
4. מחיקת משתמש מהמערכת
5. השמעת שיר מסויים מרשימת השמעה של משתמש
6. יצירת משתמש חדש במערכת

Query String - GET

● חלק מה-URL המאפשר לנו לעשות השמה לפרמטרים, בצורת מפתח-ערך

● דוגמה:

- https://www.google.com/search?q=query+string&sca_esv=986cfd32cb66a37f

